Character Recall Is Not Protection: A Document-Level Leakage Analysis of Bounded-Memory PII and Credential Redaction for Operational Logs

Mounish Senisetty*, Sandhiya Kennedy, Ashrith Reddy N, Parvathy R
Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, Kerala, India
*am.ai.u4aid23058@am.students.amrita.edu, am.ai.u4aid23056@am.students.amrita.edu,
am.ai.u4aid23014@am.students.amrita.edu, parvathyr@am.amrita.edu

***Abstract*—Automated redaction of personal data and credentials is routinely judged by character- or entity-level recall, yet a shared log file is safe only if *every* sensitive span in it is removed. We study this gap for redactors that must also run in bounded memory over multi-gigabyte logs. We first describe a boundary-safe streaming redactor whose window overlap is derived from per-detector locality bounds, so that chunked and multi-worker execution cannot cut a detection in half at a buffer seam. We then propose a leakage-centric evaluation protocol that classifies every ground-truth span as fully redacted, partially redacted or missed, and reports the document no-leak rate, a leak-tolerance curve, leak concentration and shape-stratified leak rates, together with an oracle union-of-systems ceiling. Applying the protocol to five redaction systems on 8,002 mandatory spans in 95 held-out documents shows that the best single configuration fully redacts 80.5% of spans yet leaves 82.1% of documents leaking; even the union of all five systems misses 8.2% of spans and leaves 72.6% of documents exposed. Passwords embedded in prose leak at 87–97% for every system. The redactor holds net resident memory between 8.4 and 8.54 MiB from 8 MiB to 2 GiB of input and scales to 41.1 MB/s on four workers. The results argue for reporting document-level leakage alongside recall and against using any current automatic redactor as an unsupervised release gate.**

***Index Terms*—data redaction, personally identifiable information, secret detection, data leakage, streaming algorithms, evaluation methodology**

I. Introduction

Application logs, database exports and incident captures are copied between teams, vendors and analytics platforms every day. They routinely carry personally identifiable information (PII), such as e-mail addresses, phone numbers and payment cards, alongside credentials such as API keys, bearer tokens and private keys. Credentials in particular continue to leak at scale through software artefacts [1], [2], and organisational data-loss-prevention controls [3] rarely inspect the content of bulk text before it leaves a team.

Automatic redaction tools fall into two largely separate families: PII de-identification frameworks such as Microsoft Presidio [4], and secret scanners such as gitleaks [5] and TruffleHog [6]. Both are usually evaluated with precision and recall at character or entity level. For a person deciding whether a file may be shared, however, the relevant question is binary: *did anything sensitive survive?* A redactor that masks 95% of sensitive characters can still leak something in nearly every document if the remaining 5% is spread thinly. Recall averages hide exactly this failure mode.

A second, engineering-level obstacle appears when the input is large. A multi-gigabyte log cannot be redacted as one in-memory string, so it is read in chunks. A secret that straddles a chunk boundary is then seen only in fragments, and neither fragment may match a detector. Ad-hoc overlaps reduce but do not remove this boundary leak.

This paper addresses both problems and makes the following contributions:

1) **Boundary-safe bounded-memory redaction.** We describe a streaming and multi-worker redactor in which every detector declares upper bounds on the span length it emits and the context it reads; the engine derives the minimum window overlap from these bounds, defers detections that cross the emission cut, and cross-checks worker seams with a sequential fallback (Section III).
2) **A leakage-centric evaluation protocol.** Beyond character metrics, we classify every ground-truth span as fully redacted, partially redacted or missed, and report the document no-leak rate, a k-leak tolerance curve, leak concentration, shape-stratified leak rates and an oracle union-of-systems ceiling (Section IV).
3) **An empirical leakage study.** We compare five redaction configurations from three tool families on three held-out corpora and show that document-level protection is far below what recall suggests, that combining every tool does not close the gap, and that passwords in prose are a shared blind spot (Section VI).

II. Related Work

De-identification of text. Text de-identification began in clinical informatics with dictionary and pattern systems [7], later refined by Philter [8], shared-task corpora [9] and recurrent sequence labellers [10]. Presidio [4] combines pattern recognisers with spaCy or transformer models, and generalist

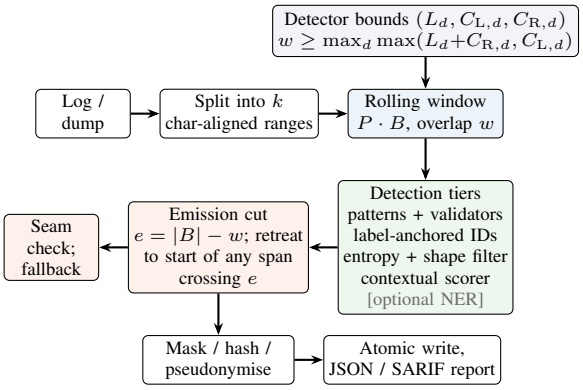


Fig. 1. Redaction pipeline. Detectors declare locality bounds from which the engine derives the window overlap w; detections crossing the emission cut are deferred to the next pass, and parallel seams are cross-checked before output is assembled.

span-extraction models such as GLiNER [11] broaden entity coverage. Lison *et al.* [12] argue that masking surface entities is not the same as anonymisation, and the Text Anonymization Benchmark [13] introduced document-level and re-identification oriented metrics for legal text. Our protocol follows the same spirit but targets mixed PII and credential leakage in operational text, and adds span outcome classes and a cross-system ceiling.

Credential detection. Secret detection moved from vendor regular expressions [14] to learned false-positive filters [15] and password classifiers [16]. SecretBench [17] standardised evaluation, and a nine-tool comparison [18] traced low precision to broad patterns and uncalibrated entropy thresholds. Named-entity models have also been adapted to security text [19]. These tools target version-controlled source trees; their behaviour on free-form logs and prose is rarely measured.

Streaming text processing. Sublinear-memory processing of unbounded inputs is studied in the data-stream model [20], and security pipelines already score continuous message streams, for example for threat indicators [21]. Such pipelines score short, self-contained messages and do not face entities split across buffers. In the redaction tools we surveyed, window overlaps are chosen by hand, and no analytical lower bound on the overlap is stated.

III. Boundary-Safe Streaming Redaction

Fig. 1 summarises the reference redactor used in this study. It is a local tool with a pure-Python reference implementation and an optional Rust core for the pattern and entropy tiers; no plaintext is sent to any network service.

A. Detection Tiers

Patterns with validators match fixed-format credentials (cloud access keys, vendor API tokens, JSON web tokens, PEM private keys) and structured PII (e-mail, phone, payment card, US SSN, IPv4/IPv6). A candidate survives only if its validator accepts it: Luhn checksums [22] for cards, range rules for SSNs, octet and hextet ranges for IP literals and E.164 structure for phone numbers. *Label-anchored identifiers* emit bare numeric record, account or licence numbers only when a domain anchor such as "Patient ID:" occurs within the preceding 96 characters. *The entropy tier* scores unstructured tokens of 20–8192 characters by Shannon entropy [23], character-class diversity and length, and a *benign-shape filter* suppresses 40- or 64-hex digests preceded by hash-like labels and inline `data:` URIs unless an assignment to a secret-like key precedes them. A *contextual scorer* adjusts confidence from a 48-character left context. An optional spaCy NER tier extracts names, organisations and places; it lies outside the boundary guarantee below because its receptive field is not statically bounded.

B. Overlap Derivation and Emission Retreat

Each detector d declares an upper bound L_d on the length of any span it emits and bounds $C_{\mathrm{L},d}$ and $C_{\mathrm{R},d}$ on the left and right context its decision may read. The streaming engine keeps a context buffer P of the last w emitted characters and a look-ahead buffer B. Each pass runs the detectors on $P \cdot B$, keeps detections that start inside B, and nominally emits up to the cut $e = |B| - w$. If a kept detection (i, j) satisfies $i < e < j$, the cut retreats to i, so the span is emitted whole in a later pass. If

$$w \;\geq\; \max_d \max(L_d + C_{\mathrm{R},d},\; C_{\mathrm{L},d})\,, \tag{1}$$

every detection emitted in a pass has its full declared context inside the window, and every detection crossing the cut is seen again intact in the next window. The streamed span set therefore equals the whole-document span set for any chunking, which removes boundary leaks for all detectors that honour their declared bounds. The engine computes (1) at start-up and rejects any smaller user-supplied overlap. For the shipped deterministic tiers the largest constants are $C_{\mathrm{L}} = 96$, $C_{\mathrm{R}} = 8$ and $L_{\max} = 8192$, and the 8192-character spans come from detectors that read no right context, so the default overlap is $w = 8192$.

C. Parallel Seams

For large files the byte stream is split into k ranges aligned to UTF-8 code points, each processed by a separate worker with w characters of read-only context on either side. A detection crossing an interior seam belongs to the left worker; the right worker starts emitting at the end of that detection. Both neighbours compute the hand-off offset independently from their own view of the seam, and the coordinator compares them before assembling output. Any disagreement triggers a sequential re-run, so parallel output is byte-identical to sequential output or falls back to it. Working memory is bounded by $O(k(w + m + c))$ characters for minimum emit size m and chunk size c, independent of input length.

D. Verification

An automated suite of 133 tests checks byte-for-byte identity between whole-document, streamed and multi-worker execution under exhaustive small-step chunkings, property-based

random inputs, entities planted across seams and injected seam disagreements that must trigger the fallback, as well as Rust–Python parity and the declared detector bounds. On all three test splits the two backends produced identical span sets.

IV. Leakage-Centric Evaluation Protocol

Let G_d be the set of ground-truth sensitive spans in document d and R_d the set of characters a system masks. Entity labels are ignored throughout: a span counts as protected if its characters are masked, whatever class the system assigned.

Character metrics. Character precision $|G \cap R|/|R|$ and recall $|G \cap R|/|G|$ are macro-averaged over documents, and F1 is their harmonic mean. They are reported for comparability with prior work.

Span outcome classes. Each span $s \in G_d$ is *fully redacted* if all its characters are in R_d, *partially redacted* if some but not all are, and *missed* otherwise. Partial redaction is counted as a leak, because a partially exposed card number or key can permit brute force or record linkage [24]. The fully redacted rate (FR) is the share of spans in the first class.

Document no-leak rate. A document is *leak-free* if every mandatory span is fully redacted. The no-leak rate is the fraction of documents that are leak-free; it is the metric a release gate would need.

k-leak tolerance curve. Because the no-leak rate is strict, we also report the fraction of documents with at most k leaked spans for $k \in \{0, 1, 2, 5, 10\}$. The curve shows whether failures are near misses that a human reviewer could fix quickly or pervasive.

Leak concentration and composition. We report the median length of leaked spans, the share of leaked characters that come from partial rather than missed spans, and the share of all leaked characters that fall in the 10% of documents that leak most.

Shape-stratified leak rate. When the corpus carries only sensitivity labels, not types, we bucket gold spans by a coarse surface shape (numeric, alphanumeric token, capitalised words, other single token, tokens containing "@", multi-word phrases) and report the leak rate per bucket.

Oracle union ceiling. We take, per document, the union of characters masked by all evaluated systems. Its leakage is an optimistic ceiling on what any ensemble of these tools can achieve without new detectors.

False alarms and uncertainty. On documents with no gold spans, the false-alarm rate (FA) is the fraction with at least one emitted span. Interval estimates use a percentile bootstrap over documents with 2,000 resamples [25], and paired differences share resample indices. For no-leak rates we also give Wilson score intervals [26].

V. Experimental Setup

Corpora. Table I lists the test splits. Each corpus was split once into development and test parts with a fixed seed, and manifests with SHA-256 digests lock the assignment. *presidio-research* is the synthetic benchmark shipped with Presidio's evaluation suite. *secrets* contains credential examples from the Nosey Parker rule catalogue, split by rule so that test vendors are unseen, plus 200 synthetic gold-free documents of UUIDs, Git hashes, SHA-256 digests, base64 images and minified JavaScript. *RedactionBench* contains multi-paragraph documents in eleven domains with character-level *mandatory* and *contextual* labels; 95 of its 100 test documents contain at least one mandatory span, for 8,002 mandatory spans with a median of 25 per document. All RedactionBench documents were inspected during error analysis before the split, so we also report a pre-exposure code revision.

TABLE I
Held-out test splits. Gold: annotated sensitive spans; gold-free: documents with no sensitive span.

Corpus	Docs	Gold	Gold-free	Bytes
presidio-research [27]	750	1,394	63	63,678
secrets (Nosey Parker [28])	313	293	97	46,595
RedactionBench [29]	100	25,592	0	1,378,267

TABLE II
Held-out character F1, fully redacted rate (FR) and false-alarm document rate (FA), with 95% bootstrap intervals for F1. +spaCy rows lie outside the boundary guarantee.

Corpus	System	Char F1	FR	FA
presidio-research	SafeStream pattern	.267 [.235,.299]	.115	.000
	SafeStream +spaCy	.702 [.679,.725]	.538	.079
	Presidio regex	.296 [.260,.333]	.123	.000
	Presidio lg	.665 [.642,.688]	.562	.000
secrets	SafeStream pattern	.842 [.736,.906]	.437	.010
	gitleaks	.604 [.430,.765]	.311	.000
	TruffleHog	.306 [.204,.446]	.270	.000
	Presidio lg	.184 [.124,.262]	.164	.433

Systems. SafeStream (the redactor of Section III) runs on the Rust core in its deterministic configuration (*pattern*) and with the spaCy `en_core_web_sm` tier (*+spaCy*). *Presidio* 2.2.355 [4] runs with patterns only (*regex*) and with the `en_core_web_sm` (*sm*) and `en_core_web_lg` (*lg*) models. *gitleaks* 8.28.0 and *TruffleHog* 3.90.8 run with their default rules; TruffleHog uses `--no-verification` so that no candidate credential is sent to a live endpoint. The two scanners are designed for Git repositories and are evaluated here out of their native domain, which is the setting in which practitioners often repurpose them for log sanitisation.

Environment. Accuracy and throughput were measured on a Linux virtual machine with a 4-vCPU Intel Xeon (2.80 GHz), 15 GB RAM and CPython 3.13. Python baselines ran single-threaded; throughput figures are means of five trials on an otherwise idle host.

VI. Results

A. Character Metrics Overstate Protection

Table II gives the conventional view. On *secrets* the deterministic tiers reach character F1 of 0.842 and a fully redacted rate of 0.437, against 0.311 for gitleaks and 0.270 for TruffleHog; the paired bootstrap differences in FR, +0.128 [+0.073, +0.183] and +0.167 [+0.079, +0.250], exclude zero.

TABLE III
OUTCOME OF THE 8,002 MANDATORY SPANS IN REDACTIONBENCH TEST DOCUMENTS. PARTIAL: SHARE OF LEAKED CHARACTERS FROM PARTIALLY REDACTED SPANS. TOP-10%: SHARE OF LEAKED CHARACTERS IN THE 10% MOST-LEAKING DOCUMENTS.

System	Full	Part.	Missed	Leaked chars	Partial share	Top-10%
SafeStream pattern	5,688	68	2,246	22,778	3.7%	59.3%
SafeStream +spaCy	6,441	64	1,497	17,122	4.8%	66.8%
Presidio lg	5,614	395	1,993	41,589	17.7%	53.7%
gitleaks	232	0	7,770	53,991	0.0%	61.0%
TruffleHog	43	1	7,958	69,913	0.0%	53.0%
Union of all five	7,048	295	659	9,214	32.2%	70.1%

Presidio has no credential recognisers and raises a false alarm on 43.3% of gold-free documents. On *presidio-research*, pattern-only systems cap at F1 below 0.30 because most gold spans are unstructured names and places; adding a statistical NER tier lifts F1 to about 0.7 for both SafeStream and Presidio.

On RedactionBench the final SafeStream configurations reach mandatory character recall of 0.682 (pattern) and 0.761 (+spaCy), against 0.420 for Presidio lg, 0.247 for gitleaks and 0.024 for TruffleHog. The document no-leak rates tell a different story: 8 of 95 documents (8.4%, Wilson 95% interval $[4.3, 15.7]$%) for SafeStream pattern, 17 (17.9% $[11.5, 26.8]$) for SafeStream +spaCy, 10 (10.5% $[5.8, 18.3]$) for Presidio lg, 2 for gitleaks and none for TruffleHog. A configuration that masks three quarters of mandatory characters thus produces a fully clean document fewer than one time in five.

B. Anatomy of Leaks

Table III decomposes the leaks. Three observations stand out.

Leaks are mostly whole misses, not boundary errors. For both SafeStream configurations fewer than 1% of spans are partially redacted and partial spans contribute under 5% of leaked characters. Presidio lg produces about six times as many partial spans (395 against 64) and draws 17.7% of its leaked characters from them, consistent with NER span boundaries that disagree with annotation boundaries. Partial redaction matters because the exposed remainder of a structured value may be enough to reconstruct it.

Leaked spans are short and word-like. The median leaked span is 7 characters for SafeStream pattern and 9 for +spaCy and Presidio lg: the length of a name, a short identifier or a password. Such spans fall below the 20-character minimum of the entropy tier and lack the fixed structure that pattern tiers rely on, so they escape detectors built for keys and tokens.

Leakage is concentrated. The 10% of documents that leak most hold 53–70% of all leaked characters. Median leaked spans per document are 11 for SafeStream pattern, 6 for +spaCy and 8 for Presidio lg, while the worst document leaks at least 313 spans for every system.

Fig. 2 shows the k-leak tolerance curve. Allowing up to two leaked spans per document raises SafeStream +spaCy from

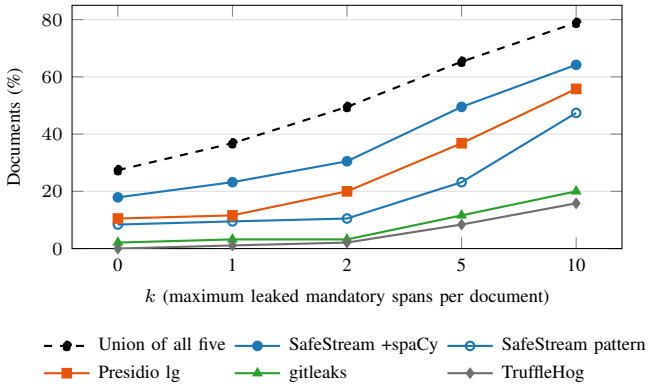


Fig. 2. k-leak tolerance curve on the 95 RedactionBench test documents with mandatory spans. $k = 0$ is the no-leak rate.

TABLE IV
LEAK RATE (MISSED OR PARTIALLY REDACTED) OF REDACTIONBENCH MANDATORY SPANS BY COARSE SURFACE SHAPE. THE CORPUS HAS NO TYPE LABELS; SHAPES COME FROM A REGULAR-EXPRESSION HEURISTIC.

Shape	Spans	SS pattern	SS +spaCy	Presidio lg	Union
Numeric (digits, punct.)	4,604	.137	.130	.114	.048
Alphanumeric token	1,619	.187	.177	.716	.143
Capitalised word(s)	1,044	.899	.267	.265	.136
Other single token	689	.576	.521	.556	.469
Contains "@"	38	.974	.947	.921	.868
Multi-word phrase	8	1.00	.500	.750	.375

17.9% to 30.5% of documents; allowing ten raises it to 64.2%. Even at $k = 10$ more than a third of documents still leak more than ten mandatory spans, so the residual is not a handful of near misses that a reviewer could patch in seconds.

C. What Leaks: Shape-Stratified Rates

Table IV shows where each tool family fails. Numeric spans make up 57.5% of mandatory spans and are handled comparably by all three PII-aware configurations (11–14% leak). The families diverge elsewhere. Pattern-only SafeStream leaks 89.9% of capitalised names, which the NER tier reduces to 26.7%. Presidio lg leaks 71.6% of alphanumeric tokens such as identifiers and keys, against 18.7% for SafeStream, reflecting the latter's credential and label-anchored tiers. The 38 spans containing "@" leak at 87–97% for every system and even for the union; since the e-mail recognisers of SafeStream and Presidio would catch addresses, these are predominantly passwords written in prose. Free-text passwords have no fixed format, often fall below entropy thresholds, and are a blind spot shared by every tool family we evaluated.

D. The Union Ceiling

Masking the union of all five systems' outputs fully redacts 88.1% of mandatory spans and leaves 659 missed, 56% fewer than the best single system. Yet only 26 of 95 documents (27.4%, Wilson $[19.4, 37.1]$%) become leak-free. The union also has the highest partial share (32.2% of leaked characters), since overlapping detections with different boundaries cover

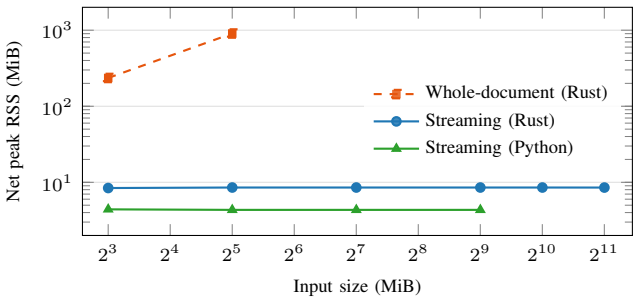


Fig. 3. Net resident memory (peak minus post-import baseline) against input size for a synthetic log dense with PII and credentials, one process per point. A whole-document run on 128 MiB did not finish within 20 minutes.

the core of a span but not its edges. Ensembling the tools in use today is therefore not a route to document-level safety; the remaining leaks require new detectors, chiefly for passwords and context-dependent identifiers.

E. Contamination and False Alarms

Because RedactionBench was inspected before splitting, the figures above are optimistic. At the last code revision before the benchmark was added, mandatory recall was 0.589 (pattern) and 0.674 (+spaCy) and the no-leak rates were 7.4% and 13.7%. Rules written after inspection raised pattern mandatory recall by +0.091 [+0.044, +0.163]; this is an upper bound on what transfers to unseen data. On point estimates, even the pre-exposure configurations exceed every Presidio configuration in mandatory recall (best 0.420).

High-entropy benign strings are the main false-alarm source for credential redaction. Before the benign-shape filter was added, the entropy tier flagged 83 of the 103 gold-free documents in the secrets development split (0.806), including every synthetic UUID, Git hash, SHA-256 digest and base64 data URI. With the filter, the test-split false-alarm rate fell from 0.794 [0.714, 0.872] to 0.010 [0.000, 0.035] and character precision rose from 0.589 to 0.857, while recall (0.829) and FR (0.437) were unchanged. The test split had been evaluated once before the filter was designed and the negatives come from the same five-category generator, so this shows suppression of those formats rather than of arbitrary production strings.

F. Resource Cost of Bounded-Memory Operation

Fig. 3 confirms that streaming memory is flat: on the Rust core net peak RSS stayed between 8.4 and 8.54 MiB as input grew 256-fold from 8 MiB to 2 GiB, and the pure-Python backend stayed between 4.3 and 4.4 MiB up to 512 MiB. Whole-document execution needed 904.6 MiB for a 32 MiB input, about 28 times the input size, because the input string, match objects and output buffer coexist.

On RedactionBench (13.8 kB average document) the Rust core sustained 10.4 ± 1.3 MB/s single-threaded, against 0.069 MB/s for Presidio regex. Enabling statistical NER slowed SafeStream by more than two orders of magnitude, to 0.047 MB/s, and Presidio sm and lg ran at 0.020 MB/s. On a 256 MB log, one, two and four workers reached 10.5 ± 0.5, 20.6 ± 0.5 and 41.1 ± 1.4 MB/s, a 3.9× speed-up with output byte-identical to the sequential run.

VII. Discussion

Report document-level leakage. The gap between 76.1% mandatory recall and a 17.9% no-leak rate arises because recall is an average over characters while the no-leak rate is a conjunction over spans. With a median of 25 mandatory spans per document, even a high per-span success rate compounds into frequent document failure. Redaction papers and tool evaluations should report the no-leak rate and a tolerance curve next to recall; otherwise improvements that matter to recall but not to release safety are indistinguishable from those that do.

Deployment implications. No evaluated tool, nor their union, is fit to release documents to untrusted parties without review. Two uses remain well supported. First, as a pre-processor for human review: masking three quarters of mandatory characters at 10 MB/s per core reduces reviewer effort, and boundary-safe chunking guarantees that splitting a file adds no new artefacts. Second, as defence in depth inside access-controlled stores, where removing most plaintext credentials from multi-gigabyte logs in fixed memory lowers exposure without risking memory exhaustion. A two-stage layout is natural: deterministic tiers inline on bulk telemetry, where their boundary guarantee holds, and the slower NER tier on free-text fields such as tickets and e-mail bodies.

Threats to validity. presidio-research and the secrets corpus are synthetic, and secret ground truth follows Nosey Parker capture groups. RedactionBench was inspected before splitting (mitigated by the pre-exposure revision) and its creators report low agreement on contextual spans, which is why we use mandatory spans only. Document-level resampling ignores clustering by rule and domain, so intervals may be too narrow. The shape buckets are a heuristic, not type labels. gitleaks and TruffleHog were run outside their repository setting and without live verification, so their results reflect domain mismatch as well as detector coverage. All measurements were taken on one machine with English input.

VIII. Conclusion

We presented a bounded-memory redactor whose window overlap is derived from declared detector locality bounds, eliminating boundary leaks in streamed and multi-worker execution, and a leakage-centric protocol for judging redactors by whether documents come out clean. On held-out data the redactor fully redacts 80.5% of mandatory spans and runs in under 9 MiB of working memory up to 2 GiB of input, but fewer than one document in five is leak-free, and the union of five tools reaches only 27.4%. Short context-dependent identifiers and free-text passwords dominate what remains. Future work will evaluate on real continuous-integration logs under a responsible-disclosure protocol, develop detectors for in-prose passwords, and seek formal context bounds for windowed neural taggers so that NER tiers can share the boundary guarantee.

Acknowledgment

The authors thank the maintainers of Presidio, Nosey Parker and RedactionBench for releasing their software and data. The large language model Claude (Anthropic) was used to help write evaluation scripts and to edit the manuscript; the authors verified all code, results and text and take full responsibility for the content.

References

[1] M. Meli, M. R. McNiece, and B. Reaves, "How bad can it git? Characterizing secret leakage in public GitHub repositories," in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2019.

[2] A. Krause, J. H. Klemmer, N. Huaman, D. Wermke, Y. Acar, and S. Fahl, "Pushed by accident: A mixed-methods study on strategies of handling secret information in source code repositories," in *Proc. 32nd USENIX Security Symp.*, 2023, pp. 2527–2544.

[3] N. Manmadhan and K. Achuthan, "Behavioural analysis for prevention of intranet information leakage," in *Proc. Int. Conf. Advances in Computing, Communications and Informatics (ICACCI)*, 2014.

[4] Microsoft, "Presidio: Data protection and de-identification SDK, version 2.2.355," https://github.com/microsoft/presidio, accessed: Oct. 2026.

[5] Z. Rice and gitleaks contributors, "gitleaks: Detect secrets in code, version 8.28.0," https://github.com/gitleaks/gitleaks, accessed: Oct. 2026.

[6] Truffle Security, "TruffleHog: Find and verify secrets, version 3.90.8," https://github.com/trufflesecurity/trufflehog, accessed: Oct. 2026.

[7] I. Neamatullah, M. M. Douglass, L.-W. H. Lehman, A. Reisner, M. Villarroel, W. J. Long, P. Szolovits, G. B. Moody, R. G. Mark, and G. D. Clifford, "Automated de-identification of free-text medical records," *BMC Medical Informatics and Decision Making*, vol. 8, p. 32, 2008.

[8] B. Norgeot, K. Muenzen, T. A. Peterson, X. Fan, B. S. Glicksberg, G. Schenk, E. Rutenberg, B. Oskotsky, M. Sirota, J. Yazdany, G. Schmajuk, D. Ludwig, T. Goldstein, and A. J. Butte, "Protected health information filter (Philter): Accurately and securely de-identifying free-text clinical notes," *npj Digital Medicine*, vol. 3, p. 57, 2020.

[9] A. Stubbs and Ö. Uzuner, "Annotating longitudinal clinical narratives for de-identification: The 2014 i2b2/UTHealth corpus," *Journal of Biomedical Informatics*, vol. 58, 2015.

[10] F. Dernoncourt, J. Y. Lee, Ö. Uzuner, and P. Szolovits, "De-identification of patient notes with recurrent neural networks," *Journal of the American Medical Informatics Association*, vol. 24, no. 3, pp. 596–606, 2017.

[11] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist model for named entity recognition using bidirectional transformer," in *Proc. Conf. North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024, pp. 5364–5376.

[12] P. Lison, I. Pilán, D. Sánchez, M. Batet, and L. Øvrelid, "Anonymisation models for text data: State of the art, challenges and future directions," in *Proc. 59th Annu. Meeting Association for Computational Linguistics (ACL)*, 2021, pp. 4188–4203.

[13] I. Pilán, P. Lison, L. Øvrelid, A. Papadopoulou, D. Sánchez, and M. Batet, "The text anonymization benchmark (TAB): A dedicated corpus and evaluation framework for text anonymization," *Computational Linguistics*, vol. 48, no. 4, pp. 1053–1101, 2022.

[14] V. S. Sinha, D. Saha, P. Dhoolia, R. Padhye, and S. Mani, "Detecting and mitigating secret-key leaks in source code repositories," in *Proc. 12th IEEE/ACM Working Conf. Mining Software Repositories (MSR)*, 2015.

[15] A. Saha, T. Denning, V. Srikumar, and S. K. Kasera, "Secrets in source code: Reducing false positives using machine learning," in *Proc. Int. Conf. COMmunication Systems & NETworkS (COMSNETS)*, 2020.

[16] R. Feng, Z. Yan, S. Peng, and Y. Zhang, "Automated detection of password leakage from public GitHub repositories," in *Proc. 44th Int. Conf. Software Engineering (ICSE)*, 2022.

[17] S. K. Basak, L. Neil, B. Reaves, and L. Williams, "SecretBench: A dataset of software secrets," in *Proc. 20th Int. Conf. Mining Software Repositories (MSR)*, 2023.

[18] S. K. Basak, J. Cox, B. Reaves, and L. Williams, "A comparative study of software secrets reporting by secret detection tools," in *Proc. ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2023.

[19] K. Simran, S. Sriram, R. Vinayakumar, and K. P. Soman, "Deep learning approach for intelligent named entity recognition of cyber security," in *Advances in Signal Processing and Intelligent Recognition Systems (SIRS 2019)*, ser. Communications in Computer and Information Science, vol. 1209. Springer, 2020, pp. 163–172.

[20] S. Muthukrishnan, "Data streams: Algorithms and applications," *Foundations and Trends in Theoretical Computer Science*, vol. 1, no. 2, pp. 117–236, 2005.

[21] K. Simran, P. Balakrishna, R. Vinayakumar, and K. P. Soman, "Deep learning approach for enhanced cyber threat indicators in Twitter stream," in *Security in Computing and Communications (SSCC 2019)*, ser. Communications in Computer and Information Science, vol. 1208. Springer, 2020.

[22] H. P. Luhn, "Computer for verifying numbers," U.S. Patent 2,950,048, 1960.

[23] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948.

[24] L. Sweeney, "k-anonymity: A model for protecting privacy," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557–570, 2002.

[25] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1994.

[26] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[27] Microsoft, "presidio-research: Evaluation and data-generation toolkit for presidio, revision 0cb3650," https://github.com/microsoft/presidio-research, accessed: Oct. 2026.

[28] Praetorian, "Nosey parker: A secrets detector, revision 2e6e7f3," https://github.com/praetorian-inc/noseyparker, accessed: Oct. 2026.

[29] S. Brynjólfsson, S. Jayakrishnan, E. Sali, D. Purwar, and M. Aggarwal, "RedactionBench," arXiv:2606.18782, 2026, dataset released under CC BY 4.0.